UC San Diego

DSC 102
Systems for Scalable Analytics

Alexander Kolovos

Topic 1: Basics of Machine Resources
Part 1: Computer Organization

Ch. 1, 2.1-2.3, 2.12, 4.1, and 5.1-5.5 of CompOrg Book

Think: Overflow?

You are working with an **8-bit signed two's complement system**.

Questions

Compute: 120+10

1. Does overflow occur?
2. What result does the machine actually produce?

Think: Overflow?

You are working with an **8-bit signed two's complement system**.

Questions

Compute: 120+10

1. Does overflow occur?

   Step 1: Convert to binary

   $[120]_{10} = [01111000]_2$ and $[10]_{10} = [00001010]_2$

   Step 2: Add

```
  01111000
+ 00001010
----------
  10000010
```

   The MSb is 1 which implies it is a negative number. In signed arithmetic, overflow occurs when you add two numbers with the same sign and get a result with the opposite sign.

   What does this result mean?

Think: Overflow?

You are working with an **8-bit signed two's complement system**.

Questions

Compute: 120+10

1. Does overflow occur?

   Addition in binary gives `10000010`. What does this result mean?

   To find the decimal value of this signed 8-bit number, we look at its two's complement:

   Bit pattern : 10000010

   Invert the bits : 01111101

   Add 1 : 01111110

   Convert to decimal : 64+32+16+8+4+2=126

   Apply the negative sign : -126

2. What result does the machine actually produce?

   The machine actually produces -126

Think: Hex Hack

A memory address is given as:

0x7F3A

Questions

1. Convert it to **binary**
2. How many **bytes** does this address represent?
3. If each address points to **1 byte**, what is the maximum addressable memory?

Think: Hex Hack

A memory address is given as:

0x7F3A

Questions

1. Convert it to **binary**

   Each hex digit = 4 bits:

   $[7]_{16} = [0111]_2$ $[F]_{16} = [1111]_2$ $[3]_{16} = [0011]_2$ $[A]_{16} = [1010]_2$

   Therefore the binary representation of 0x7F3A is 0111 1111 0011 1010

2. How many **bytes** does this address represent?

   This address represents 2 bytes.

3. If each address points to **1 byte**, what is the maximum addressable memory?

   Total addressable memory with 2 byte addresses $= 2^{16} = 65536$ or 64 KB.

Think: Hex Hack

A memory address is given as:

0x7F3A

Questions

3. If each address points to **1 byte**, what is the maximum addressable memory?

Topic 1 – Outline

- Basics of Computer Organization
  - Digital Representation of Data
  - Processors and Memory Hierarchy
- Basics of Operating Systems
  - Process Management: Virtualization; Concurrency
  - Filesystem and Data Files
  - Main Memory Management
- Persistent Data Storage

Basics of Processors

- **Processor:** Hardware to orchestrate and *execute instructions* to *manipulate data* as specified by a program
  - Examples: CPU, GPU, FPGA, TPU, embedded, etc.
- **Instruction Set Architecture (ISA):** The vocabulary of commands of a processor

Program in PL

Compile/Interpret ↓

Program in Assembly Language

Assemble ↓

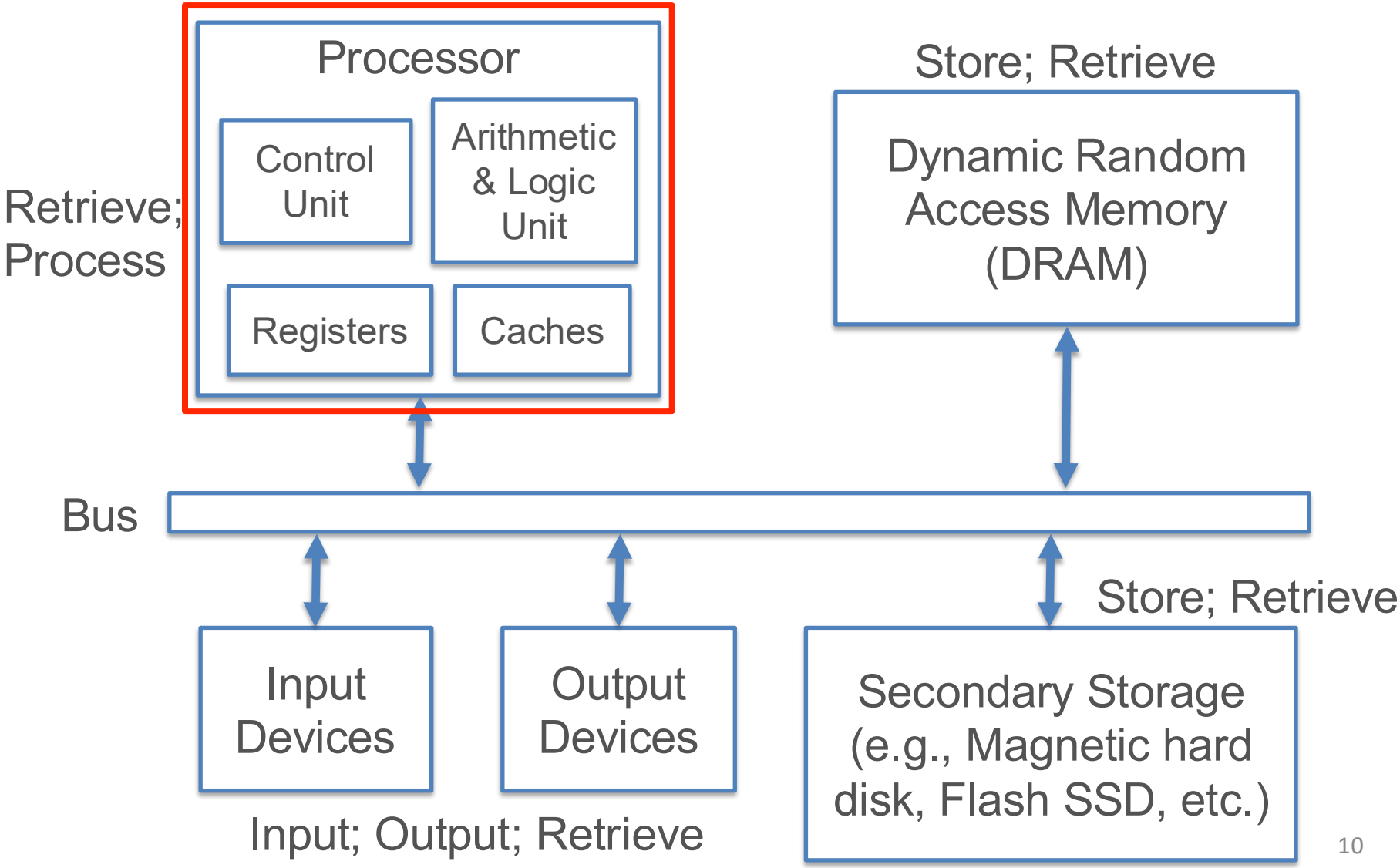
Machine code tied to ISA

↓

Run on processor

```
80483b4:       55                      push   %ebp
80483b5:       89 e5                   mov    %esp,%ebp
80483b7:       83 e4 f0                and    $0xfffffff0,%esp
80483ba:       83 ec 20                sub    $0x20,%esp
80483bd:       c7 44 24 1c 00 00 00    movl   $0x0,0x1c(%esp)
80483c4:       00
80483c5:       eb 11                   jmp    80483d8 <main+0x24>
80483c7:       c7 04 24 b0 84 04 08    movl   $0x80484b0,(%esp)
80483ce:       e8 1d ff ff ff          call   80482f0 <puts@plt>
80483d3:       83 44 24 1c 01          addl   $0x1,0x1c(%esp)
80483d8:       83 7c 24 1c 09          cmpl   $0x9,0x1c(%esp)
80483dd:       7e e8                   jle    80483c7 <main+0x13>
80483df:       b8 00 00 00 00          mov    $0x0,%eax
80483e4:       c9                      leave
80483e5:       c3                      ret
80483e6:       90                      nop
80483e7:       90                      nop
80483e8:       90                      nop
80483e9:       90                      nop
80483ea:       90                      nop
```

Abstract Computer Parts and Data



Basics of Processors

Q: *How does a processor execute machine code?*

- Most common approach: **load-store architecture**
- **Registers:** Tiny local memory ("scratch space") on processor into which instructions and data are copied
- Instruction Set Architecture specifies bit length/format of machine code commands
- Instruction Set Architecture has several commands to manipulate register contents

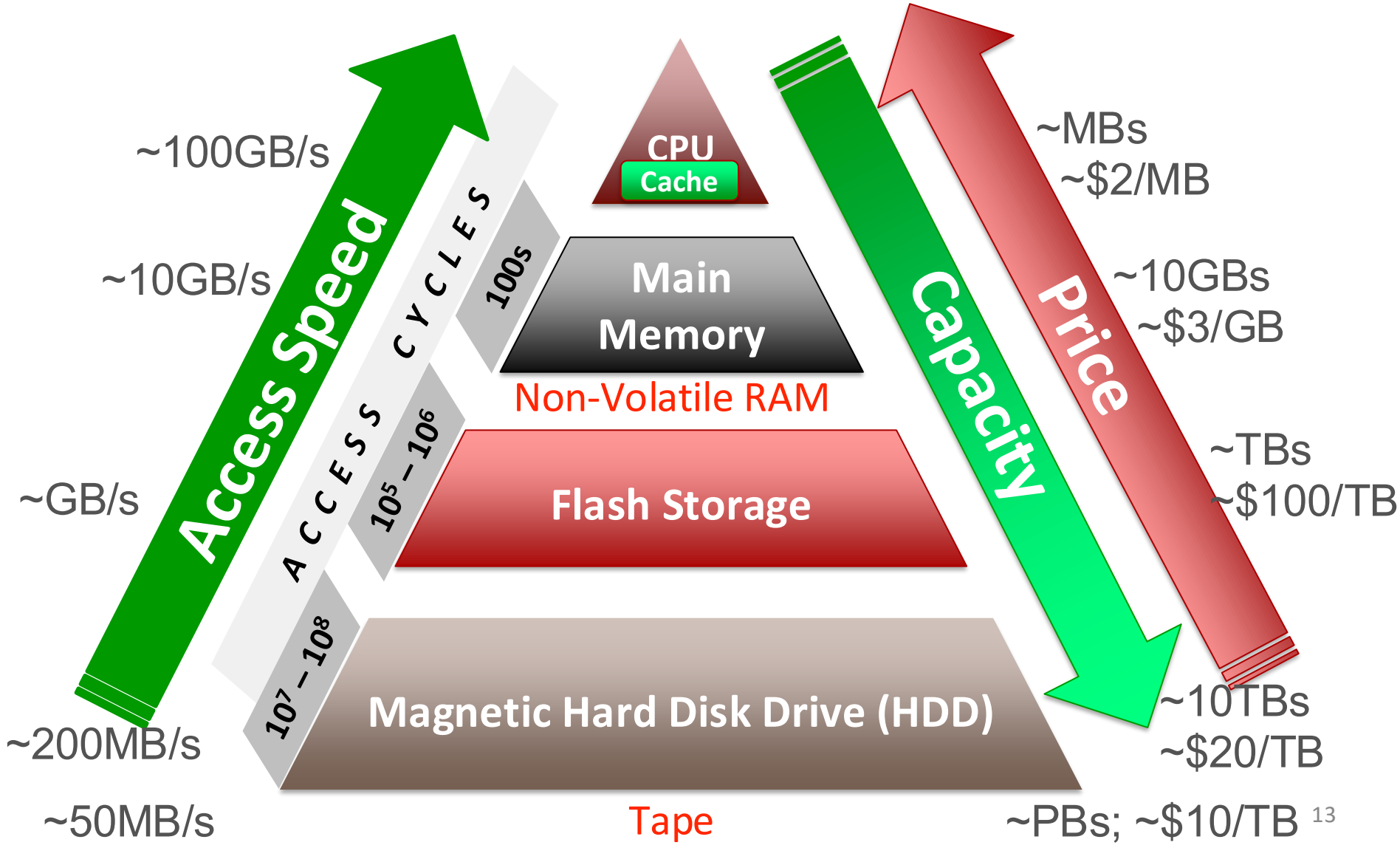
Processor Performance

***Q:** How fast can a processor process a program?*

- Modern CPUs can run millions of instructions per second!
  - ISA tells us #**clock cycles** each instruction needs
  - CPU's **clock rate** lets us convert that to runtime (ns)
- Alas, most programs do not keep CPU always busy
  - Memory access commands **stall** the processor; Arithmetic & Logic Unit and Control Unit are *idle* during memory-register transfer
  - Worse, data may not be in DRAM—wait for disk I/O!
  - So, actual *execution runtime* of program may be orders of magnitude higher than what clock rate calculation suggests

Key Principle: Optimizing access to main memory and use of processor cache is critical for processor performance!

Memory/Storage Hierarchy



Memory/Storage Hierarchy

- Typical home desktop computer today:
  - 1 TB solid state drive (SSD) / magnetic hard disk (SATA HDD)
  - 16-32 GB DRAM
  - 3-5 GHz CPU; 4-16 cores; 8 MB cache
- High-end enterprise rack server for RDBMSs:
  - 12 TB Persistent memory; 6 TB DRAM
  - 3.8 GHz CPU; 28-core per proc.; 38MB cache
- Renting on Amazon Web Services (AWS):
  - EC2 m5.large: 2-core, 8GiB: $0.115 / hour
  - EC2 m5.24xlarge: 96-core, 384 GiB, $5.53 per hour
  - EBS general SSD: $0.12 per GB-month
  - S3 store / read: $0.023 / 0.05-0.09 per GB-month

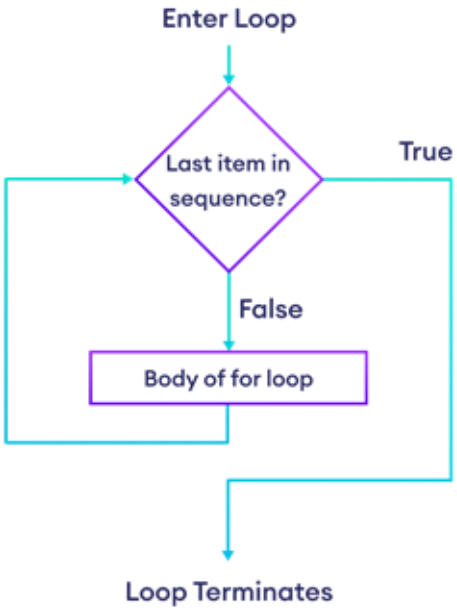
Key Principle: Locality of Reference

Carefully handling/optimizing access to main memory and use of processor cache is critical for processor performance!

- **Locality of Reference:** Many programs tend to access memory locations in a somewhat *predictable* manner
  - **Spatial:** Nearby locations will be accessed soon
  - **Temporal:** Same locations accessed again soon
- Locality can be exploited to reduce runtimes using **caching** and/or **prefetching** across all levels in the hierarchy

Temporal Locality in Programs

- If a program accesses one memory address, there is a good chance that it will access the same address again.
- Loops are excellent examples of temporal locality in programs.
  - The loop body will be executed many times.
  - The computer will need to access those same few locations of the instruction memory repeatedly.
  - Each **instruction** will be fetched over and over again, once on every loop iteration.



Temporal Locality in Data

- Programs often access the same variables over and over, especially within loops. Below, **sum** and **i** are repeatedly read and written.

```
sum = 0;
for (i = 0; i < MAX; i++)
    sum = sum + f(i);
```

- Commonly-accessed variables can sometimes be kept in registers, but this is not always possible.
  - There are a limited number of registers.
  - You can't take the address of a register (i.e., create a pointer to it)
  - There are situations where the data must be kept in memory, as is the case with shared or dynamically-allocated memory.

Spatial Locality in Programs

- If a program accesses one memory address, there is a good chance that it will also access other nearby addresses.
- Nearly every program exhibits spatial locality, because instructions are usually executed in sequence: If we execute an instruction at memory location *i*, then we will probably also execute the next instruction, at memory location *i+1*.
- Code fragments such as loops exhibit *both* temporal and spatial locality.

Spatial Locality in Data

- Programs often access data that is stored contiguously.
  - Arrays, like **a** in the code on the side, are stored in memory contiguously.
  - The individual fields of a record or object like **employee** are also kept contiguously in memory.

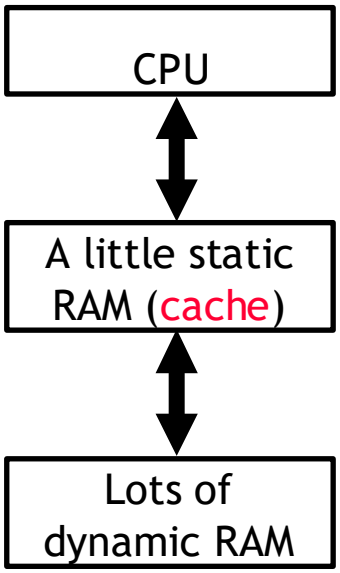
```
sum = 0;
for (i = 0; i < MAX; i++)
    sum = sum + a[i];
```

```
employee.name = "Homer Simpson";
employee.boss = "Mr. Burns";
employee.age = 45;
```

Caches And Temporal Locality

Here is how caches take advantage of temporal locality:

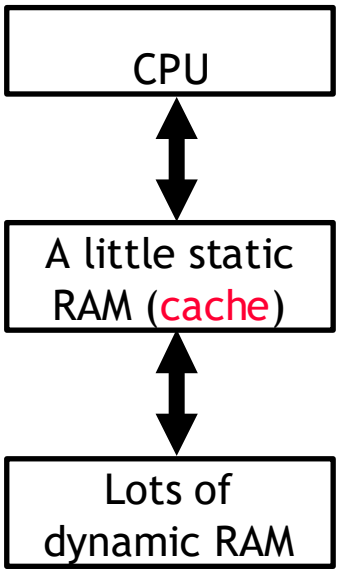
- The first time the processor reads from an address in main memory, a copy of that data is also stored in the cache.
  - The next time that same address is read, we can use the copy of the data in the cache *instead* of accessing the slower dynamic memory.
  - So the first read is a little slower than before since it goes through both main memory and the cache, but subsequent reads are much faster.
- This takes advantage of temporal locality: Commonly accessed data is stored in the faster cache memory.



Caches And Spatial Locality

Here is how caches take advantage of spatial locality:

- When the CPU reads location *i* from main memory, a copy of that data is placed in the cache.
- But instead of just copying the contents of location *i*, we can copy *several* values into the cache at once, such as the four bytes from locations *i* through *i* + 3.
  - If the CPU later does need to read from locations *i* + 1, *i* + 2 or *i* + 3, it can access that data from the cache and not the slower main memory.
  - For example, instead of reading just one array element at a time, the cache might actually be loading four array elements at once.
- Again, the initial load incurs a performance penalty, but we're gambling on spatial locality and the chance that the CPU will need the extra data.



Concepts of Memory Management

- **Caching:** Buffering a copy of bytes (instructions and/or data) from a lower level at a higher level to exploit locality
- **Prefetching:** Preemptively retrieving bytes (typically data) from addresses not explicitly asked yet by program
- **Spill/Miss/Fault:** Data needed for program is not yet available at a higher level; need to get it from lower level
  - **Register Spill** (register to cache);
  - **Cache Miss** (cache to main memory);
  - **Page Fault** (main memory to disk)
- **Hit:** Data needed is already available at higher level
- **Cache Replacement Policy:** When new data needs to be loaded to higher level, which old data to evict to make room? Many policies exist with different properties.

Think: Caching Impact on Average Access Time

Two level hierarchy: CPU cache and Memory (DRAM) only:

CPU cache access time: 10 ns
Memory access time: 1.5 microsecond
CPU cache hit rate: 0.7
Average access time therefore is: ?

How about for a three-level hierarchy:

- L1 cache on core (10ns),
- L2 cache on CPU (70ns). and
- DRAM(1.5 microsec).
- Cache hit rates on L1 is 0.7 and on L2 is 0.9.

Memory Hierarchy in Action

Q: *What does this program do when run with 'python'? (Assume tmp.csv is in current working directory)*

tmp.py

```
import pandas as p
m = p.read_csv('tmp.csv', header=None)
s = m.sum().sum()
print(s)
```

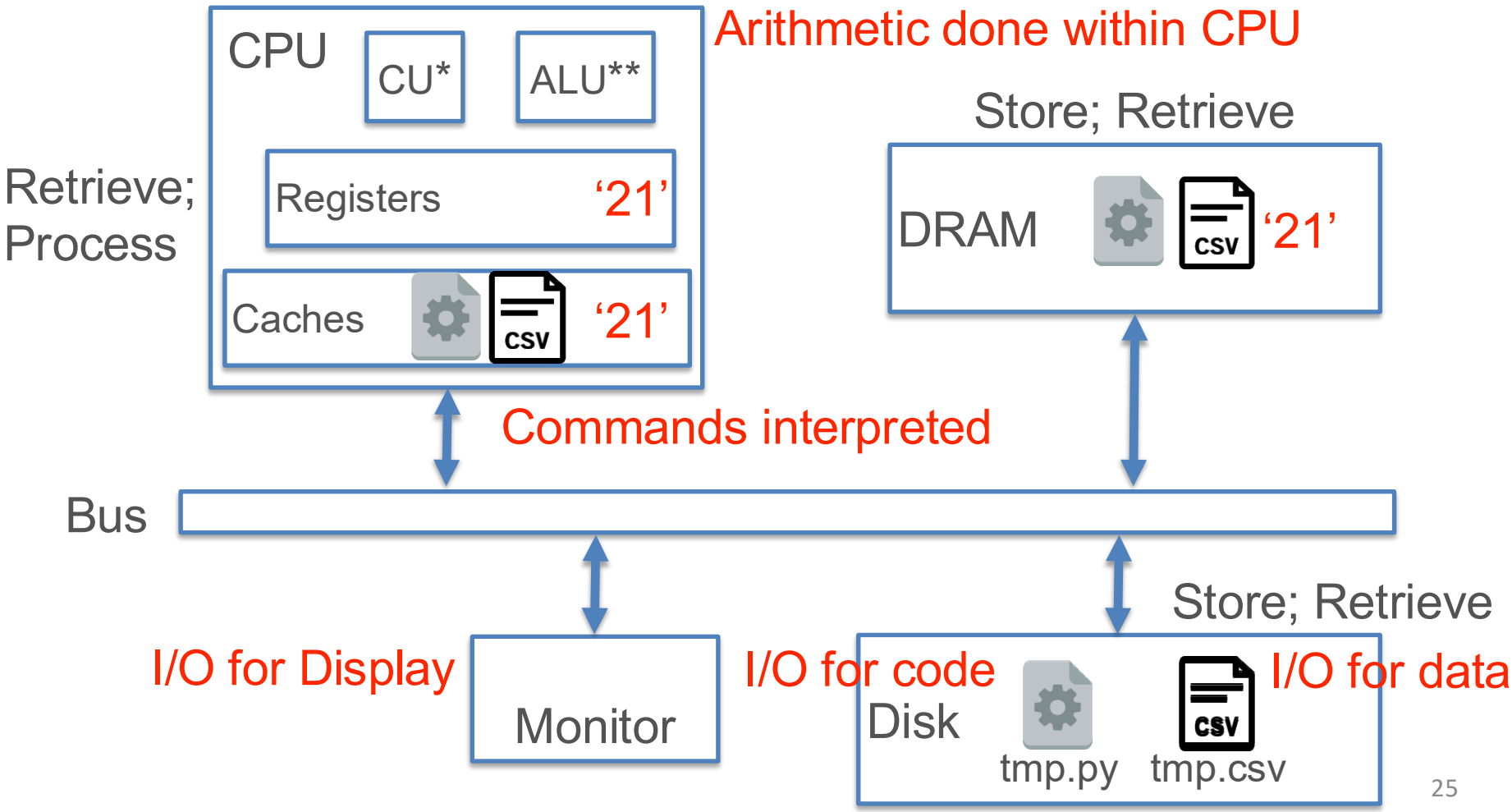
tmp.csv

```
1,2,3
4,5,6
```

Memory Hierarchy in Action

Rough sequence of events when program is executed

[*] CU: Control Unit, [**] ALU: Arithmetic Logic Unit



Locality of Reference for Data

- **Data Layout:**
  - The *order* in which data items of a complex data structure or an abstract data type (ADT) are laid out in memory/disk
- **Data Access Pattern** (of a program on a data object):
  - The *order* in which a program has to access items of a complex data structure/ADT in memory
- **Hardware Efficiency** (of a program):
  - How close *actual execution runtime* is to best possible runtime given the processor clock rate and ISA
  - Improved with careful data layout of all data objects used by a program based on its data access patterns
  - **Key Principle:** Raise cache hits; reduce memory stalls!

Locality of Reference in Data Science

- Common example: matrix multiplication (>1m cells each)
- Assume data layout in DRAM is in ***row-major*** order

$$C_{n\times m} = A_{n\times p}\ B_{p\times m}$$

DRAM A[1;] A[2;] A[3;] … B[1;] B[2;] …

Caches

```
for i = 1 to n
  for j = 1 to m
    for k = 1 to p
      C[i][j] += A[i][k] * B[k][j]
```

Slowness because of implementing fastest loop index iteration on rows in a row-major order data layout...

- Not too hardware-efficient
- Prefetching+caching means full row based on innermost loop is brought to processor cache
- A[i][.] Hits but B[k][j] Misses
- So each * op is a stall! :(

Locality of Reference in Data Science

- Common example: matrix multiplication (>1m cells each)
- Assume data layout in DRAM is in ***row-major*** order

$$C_{n \times m} = A_{n \times p} \; B_{p \times m}$$

DRAM A[1;] A[2;] A[3;] … B[1;] B[2;] …

Caches

```
for i = 1 to n
  for k = 1 to p
    for j = 1 to m
      C[i][j] += A[i][k] * B[k][j]
```

- *Logically equivalent* computation but different order of ops!
- C[i][.] and B[k][.] Hits
- A[i][k] also Hit (unaffected by j)
- Orders of magnitude fewer stalls!
- Lot more hardware-efficient

Locality of Reference in Data Science

- Common example: matrix multiplication (>1m cells each)
- Assume data layout in DRAM is in ***row-major*** order

$$C_{n \times m} = A_{n \times p} \; B_{p \times m}$$

```
for i = 1 to n
  for j = 1 to m
    for k = 1 to p
      C[i][j] += A[i][k] * B[k][j]
```

Rewrite ⇩

```
for i = 1 to n
  for k = 1 to p
    for j = 1 to m
      C[i][j] += A[i][k] * B[k][j]
```

- Although the math is the same and gives the same results ("logically equivalent"), the physical properties of program execution are vastly different
- Commonly used in *compiler optimization* and later on, also in *query optimization*

Locality of Reference in Data Science

- Matrices/tensors are ubiquitous in statistics/ML/DL programs

Q: *Would you like to write ML code in a cache-aware manner? :)*

- Decades of optimized hardware-efficient libraries exist for matrix/tensor arithmetic (*linear algebra*) that *reduce memory stalls* and *increase parallelism* (more on parallelism later) for you
  - **Multi-core CPUs:** BLAS/LAPACK (C), Eigen (C++), la4j (Java), NumPy/SciPy (Python; can wrap BLAS)
  - **GPUs:** cuBLAS, cuSPARSE, cuDNN, cuDF, cuGraph

If interested, some benchmark empirical comparisons:

https://medium.com/datathings/benchmarking-blas-libraries-b57fb1c6dc7

https://github.com/andre-wojtowicz/blas-benchmarks

https://eigen.tuxfamily.org/index.php?title=Benchmark

A Case Study

Which code performs better for sum of squares?

```
# Python loop
s = 0
for x in arr:
    s += x * x

# NumPy
s = np.sum(arr * arr)
```

And why?

What The Python Interpreter Does

```
for i in range(len(arr)):
    # 1. FETCH: Instruction pointer moves to next 'virtual' instruction [cite: 36]
    # 2. BIND: Resolve the name 'x' to a memory address
    x_obj = arr.get_item_at(i)
    # 3. TYPE CHECK (The major overhead):
    # Is 'x' an integer? A float? A custom object?
    # Python must check this for EVERY iteration[cite: 19, 192].
    type_of_x = x_obj.get_type()
    # 4. DYNAMIC DISPATCH:
    # Look up the "multiplication" method specifically for this type
    multiply_func = lookup_method(type_of_x, "__mul__")
    # 5. EXECUTE & BOX:
    # Perform the math, then wrap the raw result back into a Python Object
    temp_result_raw = multiply_func(x_obj.value, x_obj.value)
    temp_result_obj = box_into_python_object(temp_result_raw)
    # 6. REPEAT for Addition:
    # Type check 's', find its "__add__" method, perform addition, re-box 's'
    s = s.add(temp_result_obj)
    # 7. STALLS:
    # If x_obj was at a random memory address (low locality),
    # the CPU stalls and waits for DRAM.
```

The NumPy Perspective

NumPy bypasses these layers by using **homogenous data types** and **vectorization**.

- **No Type Checking:** Since the entire array is the same type (e.g., 64-bit float), NumPy checks the type **once** for the entire array rather than N times.
- **Contiguous Memory:** All values are stored in a linear block. This maximizes **row buffer locality**, meaning the memory controller can "stream" data to the CPU without the constant stalls seen in random access patterns.
- **SIMD Parallelism:** Because there is no "Instruction Pointer" bottleneck for every single number, the hardware can use SIMD (Single Instruction, Multiple Data) to square 4 or 8 numbers in a single clock cycle—effectively acting like a **Dataflow model** where operations "fire" as soon as data arrives.

Another Case Study

```
import numpy as np
import time

# Scenario: We need to add two numbers 100,000 times
iterations = 100000

# 1. Native Python Addition
start_time = time.time()
s = 0
for i in range(iterations):
    s = i + 5
python_time = time.time() - start_time

# 2. NumPy "Scalar" Addition
start_time = time.time()
s_np = np.int64(0)
for i in range(iterations):
    s_np = np.add(i, 5)
numpy_time = time.time() - start_time
```

Analyze
these behaviors.

c Analysis

- **Instruction Overhead (The "Von Neumann" Bottleneck)**
  In the NumPy version, the CPU must execute the instructions to jump into the NumPy C-library, validate that `i` can be treated as an integer, allocate a tiny bit of space for the result, and then return. In native Python, the interpreter stays within its own tight loop.
- **Lack of Vectorization:**
  NumPy is fast because of **SIMD (Single Instruction, Multiple Data)**, where it processes hundreds of numbers at once. When you call `np.add(i, 5)`, you are using a "Vector" tool to do "Scalar" work. It’s like using a massive freight train to deliver a single envelope; the overhead of starting the engine is far greater than the delivery itself.
- **Processor Stalls:**
  Every "Context Switch" from the Python interpreter to the C-extensions in NumPy can lead to minor **processor stalls**. The CPU pipeline may need to be cleared or reloaded because the control flow has jumped to a completely different area of memory.

Review Questions

- What is an ISA?
- What are the three main kinds of commands in an ISA?
- Why do CPUs have both registers and caches?
- Why is it typically impossible for data processing programs to achieve 100% processor utilization?
- Which of these layers in the memory hierarchy is the costliest: CPU cache, DRAM, flash disks, or magnetic hard disks?
- Which of the above layers is the slowest for data access?
- What is spatial locality of reference? Briefly describe a simple program that exhibits that.
- Why does data layout matter for a program's hardware efficiency?
- Which is more important for optimizing a program's hardware efficiency: data layout or data access pattern?